

Full Stack Development with MERN
Project Documentation Format Report

Book Store (BookEase)

Online Book Marketplace – MERN Stack Application

Team Members & Roles

Imran – Lead Full-Stack Developer

Technologies

MongoDB · Express.js · React · Node.js · TailwindCSS · Bootstrap

Development Environment

Visual Studio Code · Git · Node.js v24.18 · MongoDB v7.0

Generated on: July 14, 2026

1. Introduction

Project Title

The project is titled Book Store, also referred to as BookEase. It is a comprehensive, user-friendly online marketplace for discovering, buying, and selling books across multiple formats, including e-books, paperbacks, and special editions.

Team Members and Roles

- **Imran – Lead Full-Stack Developer:** responsible for database schema modeling, Express API route controllers, React user interfaces, state management, and final integration.

2. Project Overview

Purpose

Book Store is an online platform that allows users to browse, search, and purchase books with ease. The primary goal is to provide a smooth, secure shopping experience for customers, while simultaneously supporting sellers in managing inventory, listings, and order fulfillment, and administrators in overseeing platform operations, accounts, and reviews.

Key Features

- **Customer / User:** registration and login, profile updates, catalog browsing, search and filter by category/genre/author, cart management with format and quantity selection, wishlist management, secure checkout, and order confirmation and history.
- **Seller / Vendor:** registration and login, profile updates, adding and editing book listings (title, author, price, genre, description, cover image), stock management, order processing and shipping, and sales tracking.
- **Administrator:** supervision of platform activity, user and seller account management, item and order removal, seller account approval, and oversight of reviews and feedback.

3. Architecture

Frontend Architecture (React)

The frontend is a Single Page Application (SPA) built with React and bootstrapped using Vite for fast builds and hot-module replacement. Styling combines Tailwind CSS for custom utility styling with React Bootstrap for uniform, responsive components. Axios handles HTTP communication with the backend, React Router DOM manages client-side routing, and Recharts powers analytics visualizations on the admin dashboard.

Backend Architecture (Node.js & Express.js)

The backend is built with Node.js and Express.js, following the Model-View-Controller (MVC) pattern:

- **Model Layer:** uses Mongoose to define strict schema models for MongoDB collections.
- **Controller Layer:** contains core business logic, validates inputs, and executes database operations.

- **Routing Layer:** exposes REST API endpoints for user, seller, and administrator interactions.

Database Schema (MongoDB)

MongoDB is used as a document database, with schemas modeling relational structures between entities:

- **User–Book (M:M):** managed via the Interaction schema, storing reading progress, ratings, and reviews.
- **Book–Inventory (1:M):** each book document links to seller-supplied stock data.
- **User–Order (1:M):** each Order document stores purchase details linked to a User's ID.
- **Wishlist (M:M):** tracks user favorites by linking user IDs to specific book IDs.

4. Setup Instructions

Prerequisites

- Node.js: version v18 or later (v24.18 recommended)
- MongoDB: local mongod instance or MongoDB Atlas cluster connection
- Git: for version control

Installation Steps

- Clone the repository and navigate into the project directory.
- Configure the backend environment variables by creating a .env file at Backend/.env:

```
PORT=4000
MONGO_URI=mongodb://127.0.0.1:27017/BookStore
CORS_ORIGIN=http://localhost:5173
```

- Install backend dependencies and seed the default database records:

```
cd Backend
npm install
npm run seed
```

- Install frontend dependencies:

```
cd ../Frontend
npm install
```

5. Folder Structure

Client (React Frontend)

```
Frontend/
|-- src/
|   |-- Admin/      # Admin views (Ahome, Alogin, Users, Seller)
|   |-- Seller/     # Seller views (Shome, Slogin, Addbook, Orders, Editbook)
|   |-- User/       # User views (Uhome, Login, Cart, Wishlist, Profile)
|   |-- components/ # Common UI components (Navbar, Home welcome)
|   |-- config/     # Configurations (api.js instance mapping)
```

```
| |-- App.jsx      # Route configurations mapping Element elements
| +-- main.jsx    # App bootstrapping and root rendering
```

Server (Express Backend)

```
Backend/
|-- config/      # Database connection controllers (db.js)
|-- controllers/ # MVC controllers (adminController, userController, etc.)
|-- middlewares/ # Middlewares (Multer file-upload handling)
|-- models/      # MongoDB schemas (Book, User, Order, Interaction, etc.)
|-- routes/      # Express endpoints (adminRoutes, userRoutes, sellerRoutes)
|-- uploads/     # Directory hosting uploaded book cover images
|-- app.js       # Express application configuration & routing middleware
+-- server.js    # Entry server listener bootstrapping the database
```

6. Running the Application

Execute the following commands to run the MERN stack locally:

Backend Server

From the Backend directory, start the development server with nodemon:

```
cd Backend
npm start
```

Frontend Server

From the Frontend directory, start the Vite development server:

```
cd Frontend
npm run dev
```

Once both servers are running, open a browser and navigate to the local URL (default <http://localhost:5173>).

7. API Documentation

The backend exposes REST APIs on port 4000. All routes, request payloads, and operations are detailed below, grouped by user persona.

User & Reader Routes

Method	Endpoint	Description / Payload
POST	/login	Logs in a user. Payload: { email, password }. Returns a status JSON.
POST	/signup	Registers a new user. Payload: { name, email, password }.
PUT	/userprofile/:userId	Updates personal info (name, email, password) for the given userId.
GET	/item	Fetches all book listings, enriched with avgRating and salesCount.
GET	/item/:id	Fetches details of a single book by id.
POST	/interaction	Creates or updates a user's rating/review for a book.

Method	Endpoint	Description / Payload
GET	/interaction/:bookId	Fetches all reviews and the average rating for a book.
POST	/userorder	Places an order. Payload: { flatno, city, state, pincode, totalamount, bookId... }.
GET	/getorders/:userId	Retrieves order history for a specific user.
GET	/wishlist/:userId	Retrieves wishlist items for a userId.
POST	/wishlist/add	Adds a book to a user's wishlist. Payload: { userId, bookId }.
POST	/wishlist/remove	Removes a book from a user's wishlist. Payload: { userId, bookId }.

Seller & Vendor Routes

Method	Endpoint	Description / Payload
POST	/slogin	Seller login authentication. Payload: { email, password }.
POST	/ssignup	Registers a seller, pending admin approval. Payload: { name, email, password }.
POST	/items	Adds a new book. Uses Multer middleware upload.single("itemImage") for the cover image.
GET	/getitem/:userId	Fetches all books added by a specific seller (by sellerId).
PUT	/itemupdate/:id	Updates book metadata or the cover image for a given book ID.
DELETE	/itemdelete/:id	Deletes a book listing.
GET	/getsellerorders/:userId	Retrieves orders that contain books belonging to a specific seller.
PUT	/orderstatus/:id	Updates order dispatch status. Payload: { status } (e.g. Shipped).

Administrator Routes

Method	Endpoint	Description / Payload
POST	/alogin	Admin authentication login.
POST	/asignup	Registers a system admin.
GET	/users	Lists all registered customer accounts.
DELETE	/userdelete/:id	Permanently removes a user account.
DELETE	/userorderdelete/:id	Cancels and deletes an order record.
GET	/sellers	Lists all sellers, both pending and approved.
PUT	/sellerapprove/:id	Approves a pending seller account, allowing them to list books.
DELETE	/sellerdelete/:id	Permanently removes a seller account.
GET	/orders	Fetches order logs across the entire platform.

8. Authentication

Current Authentication Workflow

Authentication is handled within `UserController`, `sellerController`, and `adminController`, and currently operates as follows:

- **Registration:** the client posts details (name, email, password) to `/signup`, `/ssignup`, or `/asignup`. The server checks whether the email is already registered and, if not, saves the new user record.
- **Login Validation:** the client sends a POST request with email and password. The server looks up the email and compares the password directly:

```
if (user.password === password) {  
  return res.json({  
    Status: "Success",  
    user: { id: user.id, name: user.name, email: user.email }  
  });  
}
```

- **Authorization / Sessions:** on successful credential matching, the server returns a status message and the user's profile details. The React frontend stores this information in `localStorage` or component state to preserve the session while browsing.

Note: this workflow does not currently use password hashing or token-based sessions — see Sections 12 and 13 for the related risks and planned improvements.

9. User Interface

The interface uses modern CSS layouts combining React Bootstrap and Tailwind CSS. The primary views are described below.

Welcome Home Page

URL: `http://localhost:5173/`

Includes BookStore logo branding, a unified navigation menu with direct routes to login/signup actions, and access portals tailored for Customers, Sellers, and Administrators, alongside a promotional header introducing the platform.

Customer Product Catalog Page

URL: `http://localhost:5173/uproducts`

Displays a grid of all registered books, each showing cover art, title, author, price, and average rating. Users can select quantities and add items directly to their cart or wishlist.

Seller Dashboard View

URL: `http://localhost:5173/shome`

Displays seller-specific panels for active listings, remaining quantity, and sales data. Vendors can add new listings (with cover images via Multer), edit existing titles, and update order statuses.

Admin Control Panel View

URL: `http://localhost:5173/ahome`

Provides system metrics such as total registered users, active sellers, and global orders. Administrators can view seller and user tables, ban accounts, approve seller registrations, and delete erroneous orders.

10. Testing

Testing Strategy

Testing follows a manual User Acceptance Testing (UAT) approach combined with functional interface validation:

- **Authentication Validation:** verifies registration errors on duplicate email submissions and validation responses for invalid password combinations.
- **Inventory Cycle:** tests seller book creation, including file-upload formatting via Multer, and confirms newly added books immediately appear in the customer catalog.
- **Checkout Cycle:** verifies adding books to the cart, modifying quantities, and placing orders, and confirms that order placement decrements stock and logs correctly in the seller's order list.
- **Admin Control Cycle:** tests the seller registration approval flow, confirming that unapproved sellers cannot log in and that approval instantly enables authentication.

11. Screenshots or Demo

Demo Recording

A video walkthrough covering all key platform features — user browsing, cart operations, checkout, seller book addition, and admin approval — is available in the project workspace as Demo.mp4. The recording captures dynamic frontend transitions and real-time database state changes, verifying end-to-end functionality.

12. Known Issues

Security & Performance Constraints

- **Plain Text Passwords:** passwords are stored as plain text in MongoDB, posing a severe security risk if the database is compromised.
- **No Session Tokens:** no session validation token (such as JWT) or HTTP cookie validation is implemented; API routes do not verify auth tokens in request headers.
- **File Upload Limitations:** book cover uploads are stored directly on local server disk rather than an external CDN, limiting multi-instance horizontal scaling.

13. Future Enhancements

Proposed System Upgrades

- **Password Hashing:** integrate bcryptjs in the backend to hash passwords during signup and verify them during login.

- **Token-Based Auth:** implement JWT to secure API routes and manage user sessions securely via HttpOnly cookies.
- **Payment Gateway:** integrate a payment processing API such as Stripe or PayPal to support real-time online checkout.
- **Image CDN:** configure AWS S3 or Cloudinary storage integration for book cover images to enable scalable storage.